

A Solver

We adhere to the usual conventions for representing a game as an implementable logic program: variables are denoted by uppercase letters and predicates and function symbols by lowercase letters. An underscore (`_`) denotes a variable whose value is unused within a definition. The state of a game is represented by a situation, initially defined as a constant (e.g., `s0`). The special binary function `do(M, S)` represents the situation resulting from the execution of move `M` in situation `S`, as in the Situation Calculus. In the logical rules that follow, the symbols `if`, `and`, and `not` correspond to their logical equivalents.

A.0.1 Game-independent part

We recursively specify as a logic program all legal transitions of an extended form game from an initial situation `S` to a final situation `F` as follows:

```
game(F, F) if
    final(F).
game(S, F) if
    not final(S) and
    legal(M, S) and
    game(do(M, S), F).
```

The first two lines return the final game situation `F`, when it is reached. Otherwise, in a non-final situation `S`, the game accepts a legal move `M`, and the game continues in the next `do(M,S)` situation, until the final situation `F` is reached. To reason about what holds in individual legal situations, we use Situation Calculus represented as:

```
holds(F, S) if
    initially(F, S).
holds(F, do(M, S)) if
    effect(F, M, S).
holds(F, do(M, S)) if
    holds(F, S) and
    not abnormal(F, M, S).
```

A fluent `F` holds in the initial situation, a new fluent `F` is initiated by the effects of a move `M` executed in a situation `S`, and a fluent `F` persists after a move is made, provided it is not abnormal; abnormal fluents are terminated (do not persist). We also use rules of the form:

```
finally(F, S) if Conditions.
```

to return derived fluents `F` describing the result of the game, when the `Conditions` hold in the final situation `S`.

A.0.2 Game-dependent part

To represent a specific game we need to define game-dependent predicates for the initial state `initial/1`, the legal moves `legal/2`, what holds in the initial game situation via `initially/2`, the effects of a move on a situation via `effect/3`, what stops persisting in a situation after the execution of a move via `abnormal/3`, the final situation `final/1`, and the result of the game via `finally/2` definitions. To exemplify these definitions, we show how to describe a PD game to our solver. The initial situation `s0` is defined as:

```
initial(s0).
```

What holds in this initial situation we specify it as:

```

initially(player(p1), s0).
initially(player(p2), s0).
initially(role(p1,row), s0).
initially(role(p2,col), s0).
initially(control(p1), s0).
initially(control(p2), s0).

```

These assertions state that the two players have unique identifiers (p1 and p2), p1 plays the row role, while p2 the column role, and at the start both players have control to play next (via the `control/1` assertion, as in GDL). What holds initially changes as a result moves being made in the game. We represent moves as terms of the form `choice(P, M)`, where P is a player, and M is a move. As it is possible for any player in a Prisoner's Dilemma game to choose defect ('D') or cooperate ('C'), we write this as:

```

possible(choice(P, 'D'), S) if
    holds(player(P), S).
possible(choice(P, 'C'), S) if
    holds(player(P), S).

```

It is then legal for a player to choose a possible move if they have the control to execute it in the current situation:

```

legal(choice(P, M), S) if
    possible(choice(P, M), S) and
    holds(control(P), S).

```

When a legal move M is made by a player P, the effect that this move is actually made is recorded in the next situation:

```

effect(did(P, M), choice(P, M), S).

```

Once a legal move is executed, the player loses control, which we specify in our framework as:

```

abnormal(control(P), choice(P, M), S).

```

After a player making a choice, it is abnormal to have control, and thus cannot choose a move from that situation onwards.

Moves made in this way bring about the final situation, which we specify as a situation term with two choices made from the initial situation, one for each player.

```

final(S) if
    ground(S) and
    S = do(choice(_, _), do(choice(_, _), I)) and
    initial(I).

```

Assuming the payoff matrix of Table 2b defined as:

```

payoff('D', 'D', 1, 1).
payoff('C', 'D', 0, 5).
payoff('D', 'C', 5, 0).
payoff('C', 'C', 3, 3).

```

the outcome of the game holds information about the actual moves made by the players, and their payoffs.

```

finally(outcome(P1,M1,U1,P2,M2,U2), S) if
    final(S) and
    holds(role(P1, row), S) and
    holds(did(P1, M1), S) and
    holds(role(P2, col), S) and
    holds(did(P2, M2), S) and
    payoff(M1, M2, U1, U2).

```

We can then extract specific outcome information, e.g. the player's utility, through a goal/2 fluent (as in GDL) e.g.:

```
finally(goal(P1, U1), S) if
    finally(outcome(P1, _, U1, _, _, _), S).
finally(goal(P2, U2), S) if
    finally(outcome(_, _, _, P2, _, U2), S).
```

This completes the definition of a PD game, and allows us to use the above game description to reason about the game. If, in the specifications above, we substitute the symbol `if` with `:-`, the symbol `and` with `,`, and the symbol `not` with `\+` (representing negation by failure), we obtain an executable Prolog program that a player can query to perform reasoning within the game. For example, if player `p1` wanted to reason about the game and determine the actions needed to achieve a utility of 5, they could express this as the query shown below (queries are initiated with the symbol `'?-'`):

```
?- game(s0,F), finally(goal(p1,5),F).
```

based on the game's payoff matrix, our solver provides the following answers for `F`:

```
1 do(choice(p2,'C'),do(choice(p1,'D'),s0));
2 do(choice(p1,'D'),do(choice(p2,'C'),s0));
3 false.
```

In the first answer, `p1` acted first and `p2` second, while in the second answer, the order is reversed. This is not unexpected, as in our game definition both players have control initially, so the answers show both combinations.

B Games

Table 1 demonstrates a general game matrix, where the row player’s actions are denoted by U and D, and the column player’s actions are marked by L and R. The four possible outcomes W, X, Y, Z denote the players’ payoff π_i for the given action pairs as the tuple (π_{row}, π_{col}) .

Table 1: General payoff matrix and specific payoff matrices for considered games.

row/col	L	R
U	(W_{row}, W_{col})	(X_{row}, X_{col})
D	(Y_{row}, Y_{col})	(Z_{row}, Z_{col})

(a) A general game in matrix form.

row/col	Defect (D)	Coop. (C)
Defect (D)	$(1, 1)$	$(5, 0)$
Coop. (C)	$(0, 5)$	$(3, 3)$

(b) Prisoner’s Dilemma: $T > R > P > S$.

row/col	Hare (D)	Stag (C)
Hare (D)	$(1, 1)$	$(3, 0)$
Stag (C)	$(0, 3)$	$(5, 5)$

(c) Stag Hunt: $R > T > P > S$.

row/col	Hawk (D)	Dove (C)
Hawk (D)	$(0, 0)$	$(5, 1)$
Dove (C)	$(1, 5)$	$(3, 3)$

(d) Hawk-Dove: $T > R > S > P$.

All games in our experiment can be mapped over the general game matrix. We classify actions U and L as “Defect” denoted by D and actions D and R as “Cooperate” denoted by C . This allows us to use Axelrod’s tournaments terminology of these games in terms of four outcomes:

T (Temptation to defect): (D, C)

R (Reward for mutual cooperation): (C, C)

P (Punishment for mutual defection): (D, D)

S (Sucker’s payoff): (C, D)

C LLM-as-a-judge

To assess the utility of the LLM-as-a-judge approach, we evaluated the reasoning samples from the experiments using two LLMs: GPT-4o (gpt-4o-2024-08-06 version) and Claude 3.7 Sonnet, a state-of-the-art model not involved in generation. Each model was prompted with a simplified version of the evaluation protocol used by human judges. Their ratings were then compared against human evaluations.

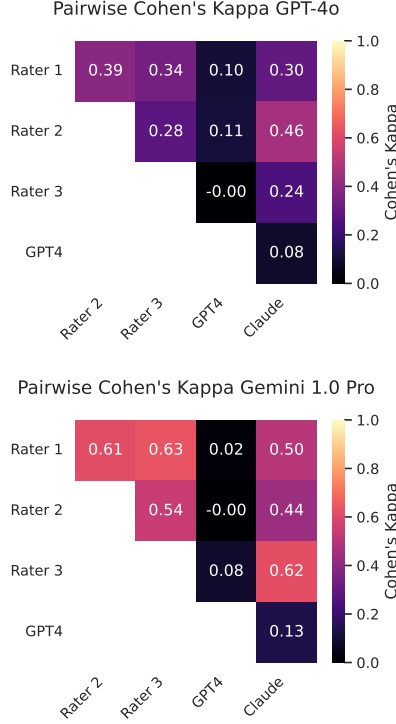


Figure 1: Pairwise inter-rater agreement. Top: GPT-4o, bottom: Gemini 1.0 Pro.

We measured agreement using Cohen’s Kappa between each LLM and the aggregated human judgment, as well as pairwise agreement among all raters (human and LLMs). Fig. 1 displays the pairwise Cohen’s Kappa values for reasoning generated by Gemini 1.0 Pro and GPT-4o. GPT-4o, when acting as a judge, showed low agreement with both human raters and Claude 3.7 Sonnet. In contrast, Claude 3.7 Sonnet exhibited substantially higher agreement with human raters—comparable to inter-human agreement.

For reasoning generated by Gemini 1.0 Pro, the agreement between GPT-4o as a judge and the aggregated human judgment was low, with Cohen’s Kappa $\kappa = 0.044$. In contrast, Claude 3.7 Sonnet showed substantially higher agreement with the aggregated human ratings, achieving $\kappa = 0.555$. For reasoning generated by GPT-4o, the agreement between GPT-4o as a judge and the aggregated human judgment was similarly low, with $\kappa = 0.159$. Claude 3.7 Sonnet again showed better alignment, with $\kappa = 0.301$.